```
import pandas as pd
import os
import warnings
from datetime import datetime, timedelta
from collections import defaultdict

# Suppress harmless openpyxl warnings
warnings.filterwarnings('ignore', category=UserWarning, module='openpyxl')

def run_calendar_automation(filepath):
    # 1. Check if file exists
    if not os.path.exists(filepath):
        print(f"Error: The file '{filepath}' was not found in {os.getcwd()}")
        return

    # --- CONDENSED: File Date Verification ---
    file_mtime = os.path.getmtime(filepath)
    file_date_str = datetime.fromtimestamp(file_mtime).strftime('%b %d, %Y at %I:%M %p')
    print(f"\n📁 Using '{os.path.basename(filepath)}' (Saved: {file_date_str}). Type 'exit' below if
this is the wrong file.")
    # ----------------------------------------

    try:
        df = pd.read_excel(filepath, header=5)
    except Exception as e:
        print(f"Error reading the Excel file: {e}")
        return

    if 'Section' not in df.columns:
        print("Error: Could not find the 'Section' column.")
        return

    df = df.dropna(subset=['Section'])

    # Filter out Waitlisted classes
    if 'Registration Status' in df.columns:
        df = df[df['Registration Status'].astype(str).str.strip().str.lower() == 'registered']
    else:
        print("Warning: Could not find the 'Registration Status' column. Proceeding without waitlist
filtering.")

    day_map = {'Mon': 'MO', 'Tue': 'TU', 'Wed': 'WE', 'Thu': 'TH', 'Fri': 'FR', 'Sat': 'SA', 'Sun': 'SU'}
    day_to_int = {'Mon': 0, 'Tue': 1, 'Wed': 2, 'Thu': 3, 'Fri': 4, 'Sat': 5, 'Sun': 6}
```

```
    parsed_events = []

    # 2. Parse all events into a list first
    for index, row in df.iterrows():
        course = str(row.get('Course Listing', ''))
        section = str(row.get('Section', ''))
        patterns_str = str(row.get('Meeting Patterns', ''))

        if pd.isna(row.get('Meeting Patterns')) or patterns_str.lower() == 'nan':
            continue

        patterns = patterns_str.split('\n\n')

        for p in patterns:
            parts = [part.strip() for part in p.split('|')]
            if len(parts) >= 5:
                try:
                    start_date = datetime.strptime(parts[0].split(' - ')[0], "%Y-%m-%d").date()
                    end_date = datetime.strptime(parts[0].split(' - ')[1], "%Y-%m-%d").date()

                    start_time = datetime.strptime(parts[2].split(' - ')[0].replace('.', '').upper(), "%I:%M
%p").time()
                    end_time = datetime.strptime(parts[2].split(' - ')[1].replace('.', '').upper(), "%I:%M
%p").time()

                    class_days = parts[1].split()
                    byday_rule = ",".join([day_map[d] for d in class_days if d in day_map])
                    target_weekdays = [day_to_int[d] for d in class_days if d in day_to_int]

                    # Adjust start_date to the first actual class day
                    adj_start_date = start_date
                    while adj_start_date.weekday() not in target_weekdays:
                        adj_start_date += timedelta(days=1)

                    parsed_events.append({
                        'course': course,
                        'section': section,
                        'start_month': start_date.month,
                        'location': ", ".join(parts[4:]),
                        'dtstart':
f"{adj_start_date.strftime('%Y%m%d')}T{start_time.strftime('%H%M%S')}",
                        'dtend':
f"{adj_start_date.strftime('%Y%m%d')}T{end_time.strftime('%H%M%S')}",
                        'until': f"{end_date.strftime('%Y%m%d')}T235959Z",
```

```
                'byday': byday_rule
            })
        except Exception as e:
            print(f"Skipping a pattern in {section} due to parsing error: {e}")

# 3. Prompt User with Retry Loop
print("\n" + "="*45)
print("            TERM SELECTION LEGEND")
print("="*45)
print(" 0 - All Terms (No filter)")
print(" 1 - January   |  5 - May       |  9 - September")
print(" 2 - February  |  6 - June      | 10 - October")
print(" 3 - March     |  7 - July      | 11 - November")
print(" 4 - April     |  8 - August    | 12 - December")
print("="*45)

while True:
    choice = input("Enter the corresponding start-month of the classes\nyou want to import (0-12) or 'exit' to cancel: ").strip().lower()

    if choice == 'exit':
        print("\nOperation cancelled by user. No calendar file was created.")
        return

    try:
        target_month = int(choice)
    except ValueError:
        print("Invalid input. Please enter a number between 0 and 12, or 'exit'.\n")
        continue

    # 4. Filter events based on the user's choice
    filtered_events = [ev for ev in parsed_events if target_month == 0 or ev['start_month'] == target_month]

    if not filtered_events:
        print(f"\nNo classes found starting in month {target_month}.")
        retry = input("Would you like to try another month or exit? (Enter 'exit' to quit, or press Enter to try again): ").strip().lower()
        if retry == 'exit':
            print("\nOperation cancelled by user. No calendar file was created.")
            return
        print("\n")
    else:
        break
```

```
    # 5. Display Organized Summary for Selection
    print("\n" + "="*45)
    print("      CLASSES READY FOR CALENDAR IMPORT")
    print("="*45)

    summary_dict = defaultdict(set)
    for ev in filtered_events:
        summary_dict[ev['course']].add(ev['section'])

    section_map = {}
    counter = 1

    for course, sections in sorted(summary_dict.items()):
        print(f"\n📘 {course}")
        for sec in sorted(list(sections)):
            print(f"  [{counter}] ↳ {sec}")
            section_map[counter] = sec
            counter += 1

    print("\n" + "="*45)

    # --- NEW: Specific Class Selection Prompt ---
    while True:
        selection = input("\nEnter 'ALL' to import everything above, or specific numbers separated
by commas (e.g., 1, 3, 4): ").strip().upper()

        if selection == 'ALL' or selection == '':
            selected_sections = set(section_map.values())
            break
        elif selection == 'EXIT':
            print("\nOperation cancelled by user.")
            return
        else:
            try:
                # Parse the comma-separated string into a list of integers
                selected_indices = [int(x.strip()) for x in selection.split(',') if x.strip()]
                selected_sections = set()
                invalid_nums = []

                # Check which numbers are valid
                for idx in selected_indices:
                    if idx in section_map:
                        selected_sections.add(section_map[idx])
```

```
                else:
                    invalid_nums.append(idx)

            if invalid_nums:
                print(f"Warning: The following numbers are invalid: {invalid_nums}. Please try
again.")
                continue
            if not selected_sections:
                print("No valid classes were selected. Please try again.")
                continue

            break # Valid selection made, exit loop

        except ValueError:
            print("Invalid input format. Please enter 'ALL' or numbers separated by commas.")
    # ------------------------------------------

    # Filter events down to ONLY the selected sections
    final_events = [ev for ev in filtered_events if ev['section'] in selected_sections]

    # 6. Final Confirmation
    confirm = input(f"\nGenerate the calendar file for {len(final_events)} selected event blocks?
(Y/N): ").strip().upper()

    if confirm != 'Y':
        print("\nOperation cancelled by user. No calendar file was created.")
        return

    # 7. Generate the ICS File
    ics_lines = [
        "BEGIN:VCALENDAR",
        "VERSION:2.0",
        "PRODID:-//Workday to Google Calendar//EN",
        "CALSCALE:GREGORIAN",
        "METHOD:PUBLISH"
    ]

    for ev in final_events:
        ics_lines.append("BEGIN:VEVENT")
        ics_lines.append(f"SUMMARY:{ev['section']}")
        ics_lines.append(f"DESCRIPTION:{ev['course']}")
        ics_lines.append(f"LOCATION:{ev['location']}")
        ics_lines.append(f"DTSTART;TZID=America/Vancouver:{ev['dtstart']}")
        ics_lines.append(f"DTEND;TZID=America/Vancouver:{ev['dtend']}")
```

```
            ics_lines.append(f"RRULE:FREQ=WEEKLY;UNTIL={ev['until']};BYDAY={ev['byday']}")
            ics_lines.append("END:VEVENT")

        ics_lines.append("END:VCALENDAR")

        output_filename = "UBCCalendar.ics"
        with open(output_filename, "w", encoding="utf-8") as f:
            f.write("\n".join(ics_lines))

        print("\n" + "-" * 50)
        print(f"Success! Generated {len(final_events)} class schedule blocks.")
        print(f"Saved to: {os.path.abspath(output_filename)}")
        print("-" * 50)

    # --- Run the Script ---
    if __name__ == "__main__":
        file_name = "View_My_Courses.xlsx"
        run_calendar_automation(file_name)
```